



Corrotinas

Prof. Alessandro Brawerman



Builders de Corrotinas

- launch

- Inicia uma nova corrotina que pode rodar de forma independente.
- Adicionando várias funções `launch()` ao seu código, várias corrotinas são executadas ao mesmo tempo.

- async

- Inicia uma nova corrotina que retorna um resultado, mas que não sabemos quanto tempo poderá demorar.
- Retorna um objeto do tipo `Deferred`, que é uma promessa de que o resultado vai estar lá quando estiver pronto.
- É possível acessar o resultado no objeto `Deferred` usando `await()`.

Corrotinas com launch

```
import kotlinx.coroutines.*

fun main() {
    runBlocking {
        launch {
            delay(1000) // Simula uma tarefa de 1 segundo
            println("Corrotina finalizada")
        }
        println("Executando fora da corrotina")
    }
    println("Executando fora do runBlocking")
}
```

Corrotinas com launch

- runBlocking
 - Bloqueia a execução da thread principal até que todas as corrotinas internas sejam concluídas. No exemplo acima, o runBlocking bloqueia a função main até que o bloco dentro dele termine.
- launch
 - Cria uma corrotina dentro de runBlocking. Mesmo que launch inicie uma corrotina de forma assíncrona, o runBlocking garante que a função main espere a conclusão de tudo antes de terminar.
- delay(1000)
 - Suspende a corrotina por 1 segundo, mas a execução fora da corrotina continua (veja que "Executando fora da corrotina" é impresso antes de "Corrotina finalizada").

Corrotinas com launch

```
import kotlinx.coroutines.*

fun main() {
    runBlocking {
        launch {
            println("Corrotina finalizada")
        }
        delay(1000) // Simula uma tarefa de 1 segundo
        println("Executando fora da corrotina")
    }
    println("Executando fora do runBlocking")
}
```

- Verifiquem a ordem de execução.

Corrotinas com launch

- Relembrando o código que medimos o tempo com as duas funções `heavyTask()`

...

```
val time = measureTimeMillis {  
    runBlocking {  
        println("Iniciando tarefas pesadas ...")  
        heavyTask1()  
        heavyTask2()  
    }  
} ...
```

```
suspend fun heavyTask1() {  
    delay(1000)  
    println("Fim da tarefa pesada 1")  
}
```

```
suspend fun heavyTask2() {  
    delay(1000)  
    println("Fim da tarefa pesada 2")  
}
```

- O código executou em cerca de 2 segundos

Corrotinas com launch

-
- Agora comparem com o código abaixo que cria corrotinas e paraleliza o código.

```
import kotlin.system.*
import kotlinx.coroutines.*

fun main() {
    println("Iniciando programa ...")
    val time = measureTimeMillis {
        runBlocking {
            println("Iniciando tarefas pesadas ...")
            launch { heavyTask1() }
            launch { heavyTask2() }
        }
    }
}
```

Corrotinas com launch

```
println("Execution time: ${time / 1000.0} seconds")
println("Fim do programa")
}
suspend fun heavyTask1() {
    delay(1000)
    println("Fim da tarefa pesada 1")
}
suspend fun heavyTask2() {
    delay(1000)
    println("Fim da tarefa pesada 2")
}
```

- O código executou em cerca de 1 segundo

Corrotinas com async

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    val result1: Deferred<Int> = async { heavyTask(1) }
    val result2 = async { heavyTask(2) }

    println("Resultado 1: ${result1.await()}")
    println("Resultado 2: ${result2.await()}")
}

suspend fun heavyTask(num: Int): Int {
    delay(2000) // Simula uma tarefa demorada
    return num
}
```

async cria corrotinas que podem retornar um valor.

await espera pela conclusão e retorna o resultado.

Note que result1 e result 2 são objetos Deferred do tipo Int, representando um resultado que será obtido de forma assíncrona.

Especificar isto é opcional devido à inferência de tipo em Kotlin, tanto que result2 não está explícito.

Decomposição Paralela

-
- A decomposição paralela envolve dividir um problema em subtarefas menores que podem ser resolvidas em paralelo.
 - Quando os resultados das subtarefas estiverem prontos, é possível combiná-los em um resultado final.
 - Para isto é preciso um novo conceito, o `coroutineScope{}`.

coroutineScope()

-
- O `coroutineScope{}` é uma função em Kotlin que cria um escopo local para a tarefa ou tarefas a serem iniciadas.
 - As corrotinas iniciadas neste escopo são agrupadas dentro deste escopo.
 - Ele suspende a execução até que todas as corrotinas dentro desse escopo sejam concluídas.
 - Ao contrário de `runBlocking`, que bloqueia a thread atual, `coroutineScope` não bloqueia e é usado dentro de corrotinas suspensas.
 - Coloca todas as corrotinas em uma única operação síncrona, dentro do escopo criado.

coroutineScope()

```
import kotlinx.coroutines.*
fun main() {
    runBlocking {
        coroutineScope {
            launch {
                delay(1000)
                println("Corrotina 1 concluída") }
            launch {
                delay(500)
                println("Corrotina 2 concluída") }
        }
        println("Todas as corrotinas terminaram") }
    println("Thread principal terminou")
}
```

coroutineScope()

- Voltando ao exemplo das tarefas pesadas.

```
fun main() {  
    println("Iniciando programa ...")  
    val time = measureTimeMillis {  
        runBlocking {  
            println("Iniciando tarefas pesadas ...")  
            coroutineScope{  
                launch { heavyTask1() }  
                launch { heavyTask2() }  
            }  
        }  
    }  
    println("Execution time: ${time / 1000.0} seconds")  
    println("Fim do programa")  
}
```

coroutineScope()

```
import kotlin.system.*
import kotlinx.coroutines.*

fun main() {
    println("Iniciando programa ...")
    val time = measureTimeMillis {
        runBlocking {
            println("Iniciando tarefas pesadas ...")
            coroutineScope{
                launch { heavyTask1() }
                launch { heavyTask2() }
            }
        }
    }
    ...
}
```